

# Trabajo: Técnicas de Reconocimiento de Patrones

Jorge Bengoa Pinedo

Bogurad Barański Barańska

Sergio Izquierdo Morona

7 de febrero de 2026

## Índice

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>1. Introducción a la clasificación de imágenes térmicas</b> | <b>2</b>  |
| 1.1. Redes Convolucionales...                                  | 2         |
| 1.2. Metodología Yolo                                          | 2         |
| <b>2. Fundamentos de funcionamiento de la red Yolo</b>         | <b>3</b>  |
| 2.1. Neurona de la red Yolo                                    | 3         |
| 2.2. Topología de la red                                       | 5         |
| 2.3. Reglas de entrenamiento                                   | 11        |
| <b>3. Desarrollo</b>                                           | <b>11</b> |
| <b>4. Resultados</b>                                           | <b>11</b> |

# **1. Introducción a la clasificación de imágenes térmicas**

Usamos la version YOLO v5 [1] Explicar el estado del arte un poco de que hay y tal.... redes tal, este otro tipo de clasificación

## **1.1. Redes Convolucionales...**

## **1.2. Metodología Yolo**

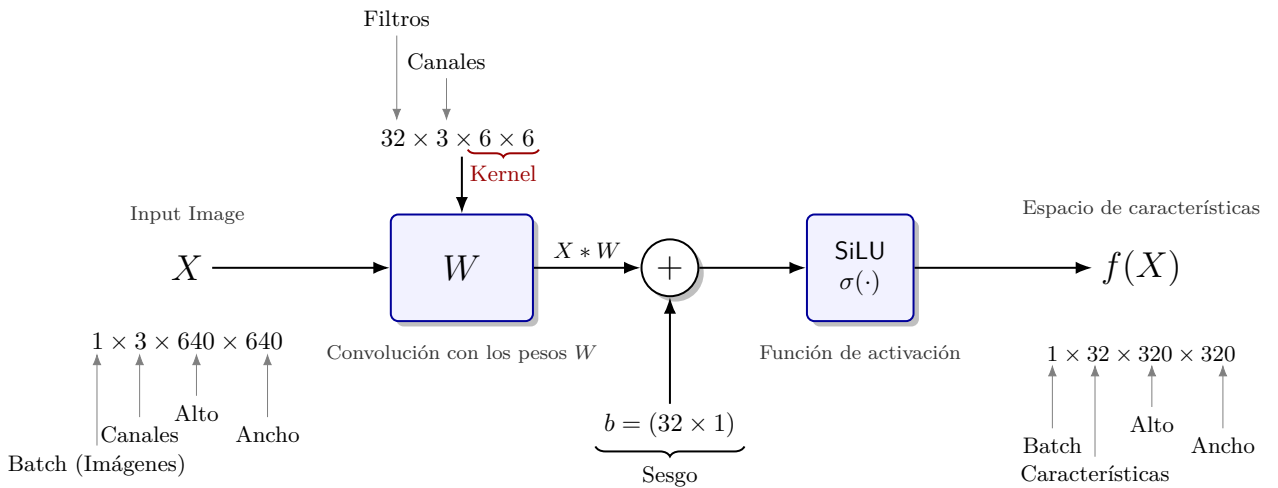
## 2. Fundamentos de funcionamiento de la red Yolo

A continuación se desarrolla el funcionamiento de la red YOLO mediante la teoría de Marr para lo cuál es necesario describir el nivel implementacional y algorítmico. En este caso las redes neuronales artificiales se componen de 3 componentes que se describirán a continuación:

- Caracterización de la neurona.
- Definición de una topología de interconexión.
- Definición de unas reglas de entrenamiento.

### 2.1. Neurona de la red Yolo

La unidad fundamental de procesamiento en la arquitectura YOLO es la capa convolucional (típica de las CNN). En esta sección se describe el funcionamiento de dicha operación, tomando como caso de estudio la capa de entrada, analizando tanto el entrenamiento específico realizado para imágenes térmicas como la implementación por defecto, tal como se ilustra en la Figura 1.



**Figura 1:** Estructura de la neurona a la entrada de la red YOLO.

La operación de esta neurona convolucional está definida por los siguientes hiperparámetros:

- **Número de filtros (Características  $C_{out}$ ):** Determinado por la primera dimensión del tensor de pesos. Define cuántos mapas de características se generarán.
- **Canales de entrada ( $C_{in}$ ):** Determinado por la segunda dimensión del tensor de pesos (e.g., 3 para RGB, 1 para imágenes térmicas).
- **Tamaño del Kernel ( $k$ ):** Define las dimensiones espaciales de la ventana de convolución (últimas dos dimensiones de los pesos). En la arquitectura YOLO analizada, se emplean principalmente los tamaños (6,6) para la entrada, y (3,3) o (1,1) para capas profundas.
- **Padding ( $P$ ):** Parámetro de relleno utilizado para controlar las dimensiones espaciales de la salida e indicar los bordes al llenarlos de ceros. En YOLO se utilizan típicamente valores de (2,2), (1,1), o (0,0).
- **Stride ( $S$ ):** Controla el paso o desplazamiento del filtro sobre la imagen. Un  $S = 1$  mueve el filtro píxel a píxel (preservando resolución), mientras que  $S = 2$  lo desplaza cada dos píxeles, reduciendo la dimensión espacial a la mitad.
- **Dilatación ( $d$ ):** Controla la separación entre los puntos que analiza el kernel (también conocida como *atrous convolution* cuando  $d > 1$ ). En esta implementación de YOLO, el parámetro es (1,1), lo que indica una convolución estándar compacta sin huecos entre los elementos del kernel.

- **Agrupación ( $g$ ):** Controla la conectividad entre los canales de entrada y salida. Dado que en YOLO se utiliza  $g = 1$ , se realiza una convolución estándar donde todos los canales de entrada contribuyen al cálculo de cada filtro de salida.

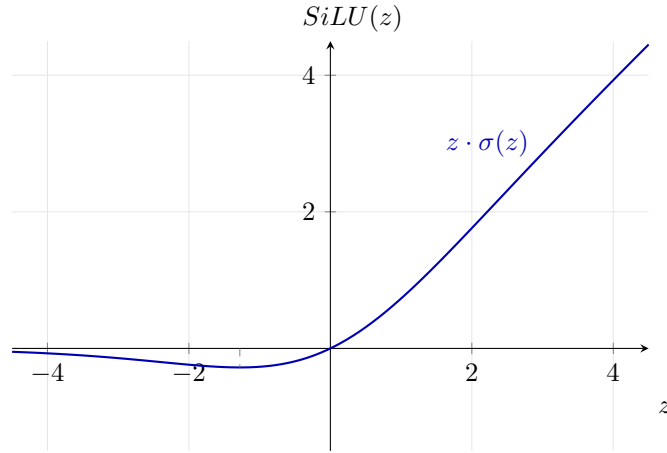
A partir de estos hiperparámetros, es posible determinar las dimensiones espaciales del mapa de características de salida. Dado que se contempla el factor de dilatación, se debe utilizar la relación generalizada descrita por [2].

$$\text{Salida} = \left\lfloor \frac{\text{Entrada} + 2 \cdot P - [k + (k - 1)(d - 1)]}{S} \right\rfloor + 1 \quad (1)$$

En cuanto a la activación de cada neurona, el proceso consiste en aplicar la función SiLU (*Sigmoid Linear Unit*) al resultado de la operación lineal (convolución más sesgo). Matemáticamente, la activación  $f(X)$  se expresa como:

$$f(X) = \underbrace{[(W * X) + b]}_z \cdot \underbrace{\sigma([(W * X) + b])}_z \quad (2)$$

donde  $\sigma(z) = \frac{1}{1+e^{-z}}$  es la función sigmoide logística. Se utiliza esta función de activación porque permite el cálculo de gradientes y solo permite valores negativos pequeños para mejorar el flujo de gradientes [3, 4], se ilustra en la Figura 2.



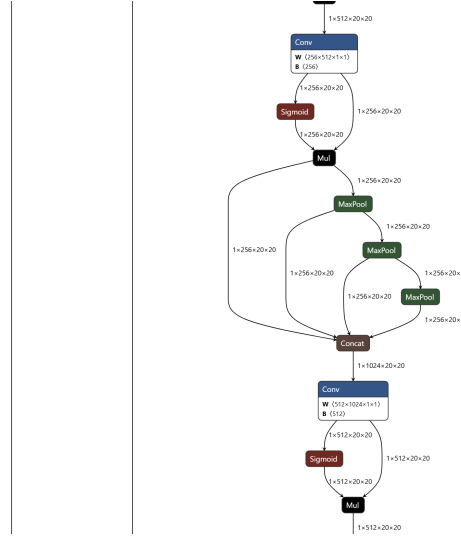
**Figura 2:** Representación gráfica de la función de activación SiLU.

Por otro lado, la operación de convolución discreta 2D ( $W * X$ ) en el contexto de aprendizaje profundo se implementa técnicamente como una correlación cruzada (sin voltear el kernel) pues solo se busca conocer los pesos y no aplicar un filtro con una topología concreta. Para un filtro de salida específico ( $c_{out}$ ) en la posición espacial ( $h, w$ ), la operación se define formalmente como [2]:

$$(W * X)(c_{out}, h, w) = \sum_{l=0}^{\frac{c_{in}}{g}-1} \sum_{i=0}^{K_H-1} \sum_{j=0}^{K_W-1} W(c_{out}, l, i, j) \cdot X(l, h \cdot S + i \cdot d, w \cdot S + j \cdot d) \quad (3)$$

## 2.2. Topología de la red

En esta sección se discute la topología de la red tras analizar los artículos principales de los creadores de Yolo [5][6][7][8] y su diagrama de conexión mediante la herramienta *Netron* donde se introduce la red en formato *.onnx* para su análisis, un ejemplo de parte de la red se puede ver en la figura 3.



**Figura 3:** Ejemplo de parte de la red Yolo en la aplicación *Netron*.

Uno de los bloques que más se repite a lo largo de la arquitectura de la red Yolo es el bloque ResNet, cuya estructura se encuentra representada en la figura 4. La filosofía detrás del diseño de este bloque viene detallada en el artículo [9], y su objetivo principal se mejoró el rendimiento de las redes neuronales profundas cuando se acumulan múltiples capas.

La problemática surge durante el entrenamiento: al acumular múltiples capas consecutivas, los gradientes tienden a explotar. Antes de la existencia de este bloque, esto se mitigaba mediante la normalización en capas intermedias, pero aun así se observaba una degradación en la convergencia. Para entender esta degradación, se puede entender la operación de convolución como una aplicación identidad con cierto ruido superpuesto que representa las características a extraer. Por tanto, mediante una conexión lineal directa se optimiza el comportamiento del método de optimización utilizado, mejorando así la convergencia.

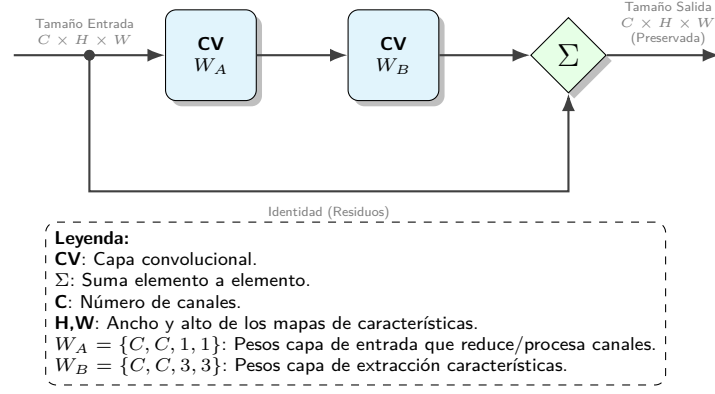
La premisa de que los bloques se comportan como aplicaciones lineales con ruido se demuestra empíricamente en el propio artículo [9]. Aunque se trata más de una heurística que de un argumento formal, en casos reales, como el de la figura 5, presenta un mejor comportamiento que simplemente acumular capas, y solo sería problemático si el objetivo del bloque fuera aproximar la función nula.

La interpretación del bloque ResNet como una aplicación lineal con ruido se deduce fácilmente a partir de las ecuaciones del modelo:

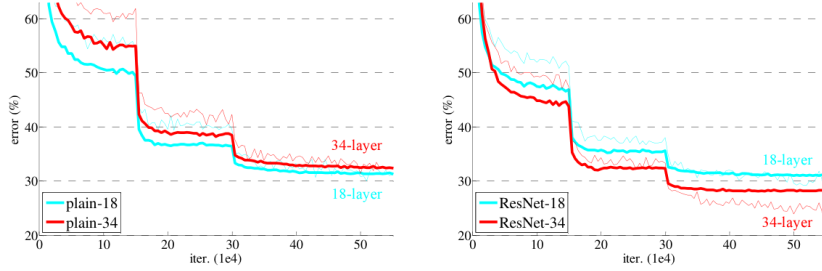
$$y = \mathcal{H}(x) = \mathcal{F}(x) + x \quad (4)$$

donde  $y$  es la salida de aplicar  $\mathcal{H}$  a la entrada  $x$  que se descompone entre una función que representa la convolución  $\mathcal{F}(x)$  y la aplicación lineal directa. Por tanto, se puede asemejar el funcionamiento de las capas convolucionales como un extractor del ruido o características.

Profundizando en el detalle del propio bloque, este está formado por dos capas, donde la primera se encarga de procesar los canales para que la segunda simplemente extraiga las características de los anteriores.



**Figura 4:** Arquitectura de un bloque ResNet.



**Figura 5:** Representación del error de entrenamiento en función del número de capas. A la izquierda se representa una arquitectura con capas apiladas y a la derecha una arquitectura de capas apiladas utilizando el bloque ResNet. Figura extraída de [9].

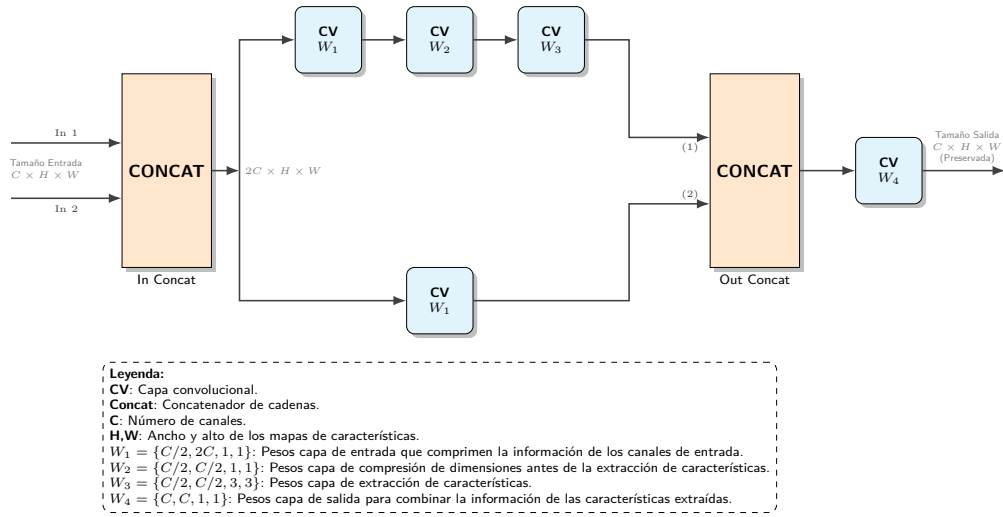
El siguiente bloque básico dentro de la arquitectura Yolo es el bloque CSP o C3. La filosofía detrás del bloque representado en la figura 6 se basa en el artículo [10]. Este componente, se diseña para mejorar la ineficiencia dentro de las redes convolucionales asociada a la duplicidad de información en el gradiente. Esta problemática, se debe a que los gradientes de retropropagación se copian y repiten a través de las diferentes capas, lo que genera cálculos redundantes que no contribuyen significativamente al aprendizaje de características nuevas. Para mitigarlo la arquitectura CSP envía el mapa de características de entrada a través de los canales base  $C$  para procesarlo en dos ramas en paralelo.

Como se observa en la figura 6, tras la concatenación de entrada, el flujo se bifurca. Una rama principal procesa la información mediante una secuencia de capas convolucionales (representadas por los pesos  $W_1$ ,  $W_2$  y  $W_3$ ), mientras que una rama parcial conecta directamente la entrada con el final del bloque a través de una única capa (peso  $W_1$  inferior). Esta división permite que el gradiente se propague por dos caminos distintos, aumentando la variabilidad del aprendizaje y reduciendo la cantidad de operaciones. Por tanto, dada una entrada  $x$ , la red procesa obtiene el tensor de salida  $y$  como la concatenación de las dos ramas en paralelo:

$$y = \mathcal{T}[\mathcal{F}(x) || \mathcal{G}(x)] \quad (5)$$

donde  $\mathcal{T}$  es la convolución de la capa de salida que sirve para fusionar la información proveniente de los dos canales concatenados  $\mathcal{F}(x) || \mathcal{G}(x)$ , la transformación  $\mathcal{F}(x)$  realizada por la capa convolucional  $W_1$  sirve para fusionar la información proveniente de los dos canales de entrada y la transformación  $\mathcal{G}(x)$  consiste en extraer características mediante la capa convolucional  $W_3$  que trabaja con un mapa de características comprimido en la capa  $W_2$ .

Por tanto, este bloque es una forma de poder reciclar el cálculo de gradientes de compresión de canales para poder tener una capa especializada únicamente en la extracción de características  $W_3$ . Para finalmente en la capa  $W_4$  combinar el contexto global de la imagen proveniente de la rama inferior con las características extraídas en la capa superior.



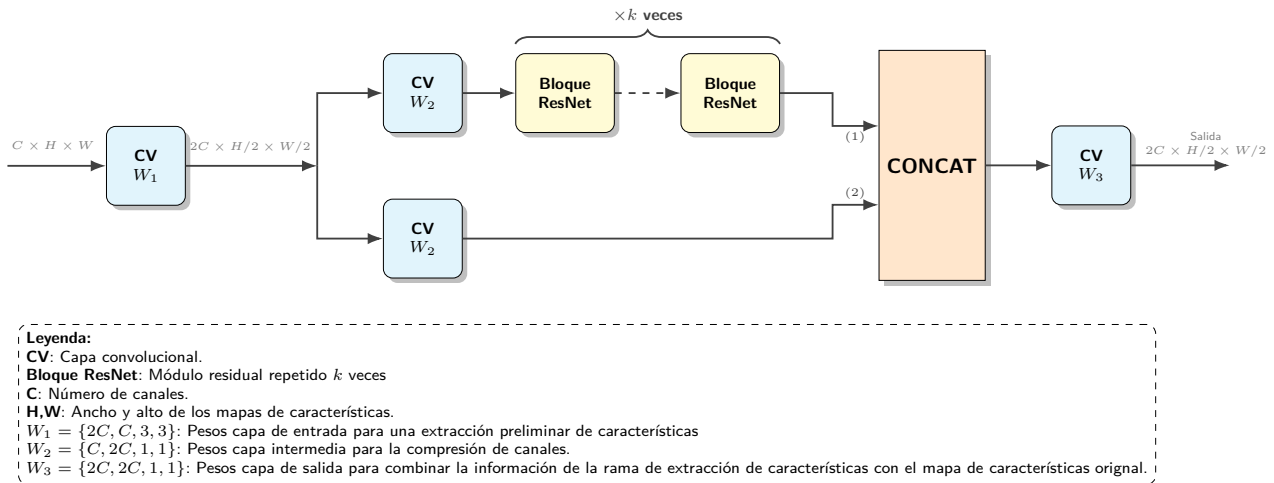
**Figura 6:** Arquitectura del bloque CSP o C3.

Una variante fundamental de la arquitectura CSP es el bloque CSPResNe(X)t o C3-k, el cual sustituye las secuencias convolucionales estándar por bloques ResNet para potenciar la extracción de características. Su representación esquemática se detalla en la figura 7, y su diseño se fundamenta nuevamente en el artículo [10].

La filosofía de este diseño reside en la concatenación en serie de bloques ResNet, repetidos un número  $k$  de veces según la complejidad de las características a modelar. Esto convierte al bloque C3-k en el mecanismo principal para controlar la profundidad de la red Yolo. Al concentrar la mayor densidad de procesamiento, este módulo se especializa en la extracción profunda de características.

En consecuencia, la integración de los módulos ResNet resulta crítica para la estabilidad del entrenamiento. Como se ha discutido previamente, las conexiones residuales mitigan el desvanecimiento del gradiente, permitiendo que el modelo converja y aprenda adecuadamente incluso al aumentar la profundidad de la red.

Analizando el esquema, es relevante destacar la función específica de cada etapa: la capa de entrada  $W_1$  actúa como un primer extractor de características superficiales, generando dos proyecciones del canal de entrada (reduciendo la resolución del espacio de características). Posteriormente, las capas  $W_2$  se encargan de comprimir estos canales: una rama profundiza en la extracción mediante la serie de  $k$  bloques ResNet, mientras que la otra preserva la señal original. Finalmente, tras la concatenación, la capa  $W_3$  fusiona ambos flujos, integrando los detalles finos aprendidos por la red profunda con la estructura general preservada por la rama inferior.



**Figura 7:** Arquitectura del Módulo CSPResNe(X)t o C3-k.

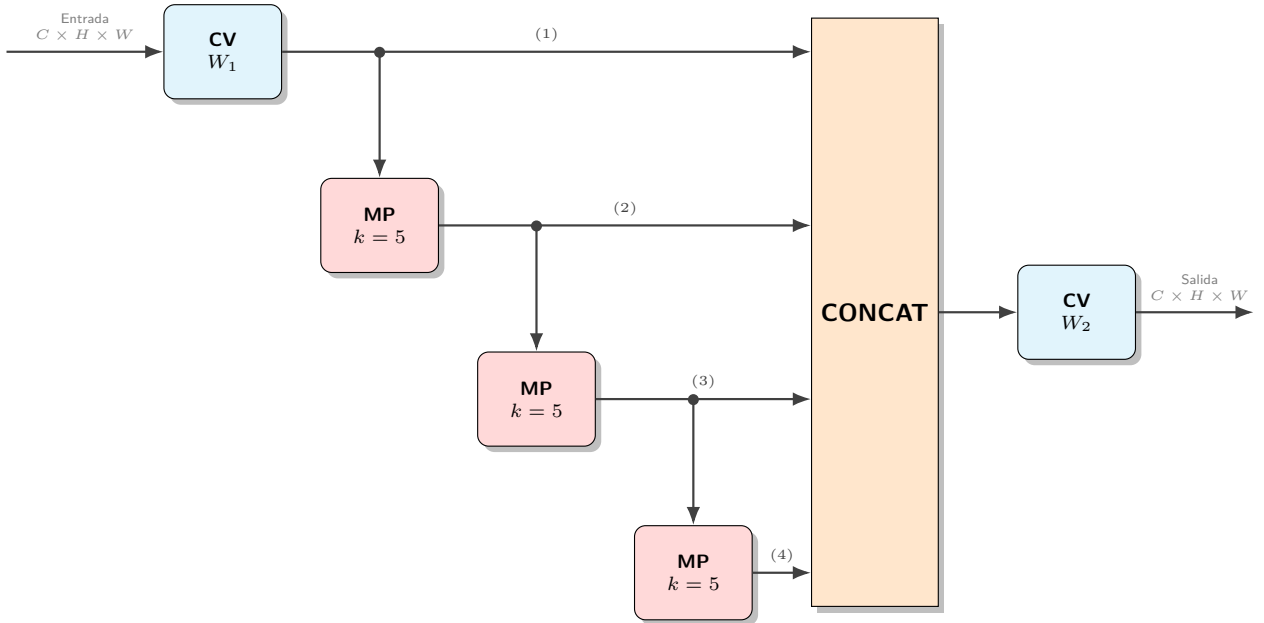
El siguiente bloque relevante en Yolo es el representado en la figura 8 que es una variación optimizada del módulo SPP descrito en el artículo [11]. Esta variación optimizada se denomina *Spatial Pyramid Pooling Fast* (SPPF). Este módulo, introducido en la arquitectura Yolo [1], tiene como objetivo principal permitir que la red analice simultáneamente el contexto global de la imagen y los detalles locales mediante una estructura en cascada que optimiza la velocidad de procesamiento.

La filosofía original del SPP [11] propone procesar el mapa de características mediante ventanas de pooling en paralelo con diferentes tamaños de kernel ( $k = \{1, 5, 9, 13\}$ ) para capturar contextos de distinto tamaño. Sin embargo, la variante SPPF logra un resultado matemático equivalente mediante una estructura en cascada más eficiente. En lugar de ejecutar múltiples ramas paralelas, pasa la entrada secuencialmente a través de varias capas de Max Pooling de tamaño  $5 \times 5$ . El Max Pooling se define como el valor máximo dentro de una ventana deslizante sobre el mapa de características:

$$y_{i,j} = \max_{(m,n) \in \mathcal{R}} x_{i+m,j+n} \quad (6)$$

donde  $\mathcal{R}$  es la región del kernel. El Max Pooling aporta invarianza a pequeñas traslaciones y deformaciones. Al quedarse solo con la característica más representativa de una región, el modelo se vuelve robusto ante ligeros desplazamientos del objeto, permitiendo reconocer patrones independientemente de su posición exacta dentro de la ventana.

Para ello, el proceso comienza con la capa convolucional  $W_1$ , que reduce los canales de entrada a la mitad con el fin de disminuir el coste computacional de las operaciones de pooling subsiguientes. Esta etapa da paso a una estructura secuencial en cascada, donde la salida de cada bloque se bifurca iterativamente: una rama alimenta al siguiente bloque y otra se dirige directamente al concatenador. Finalmente, la capa convolucional  $W_2$  integra todas estas proyecciones multiescala, generando una representación unificada y robusta frente a las transformaciones del espacio de características de entrada.



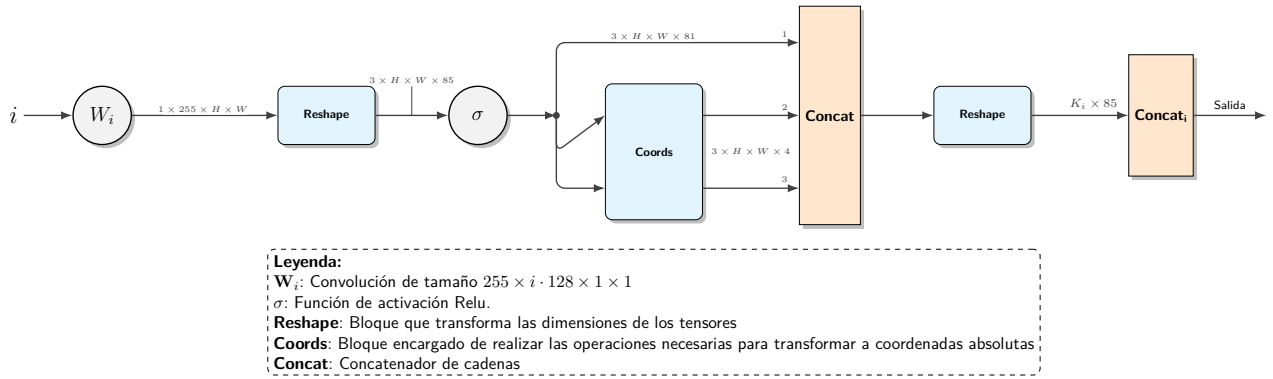
**Leyenda:**  
**CV:** Capa convolucional.  
**Concat:** Concatenador de cadenas.  
**MP:** MaxPool ( $k = 5$ ).  
**C:** Número de canales.  
**H,W:** Ancho y alto de los mapas de características.  
 $W_1 = \{C/2, C, 1, 1\}$ : Capa convolucional de entrada para comprimir la información de los canales.  
 $W_2 = \{C, 2C, 1, 1\}$ : Capa convolucional de salida para obtener la información combinada des Max pooling.

**Figura 8:** Arquitectura del bloque SPPF.

La estructura de la cabeza 9 puede explicarse directamente a través de los artículos de Yolodsaaaaa asd

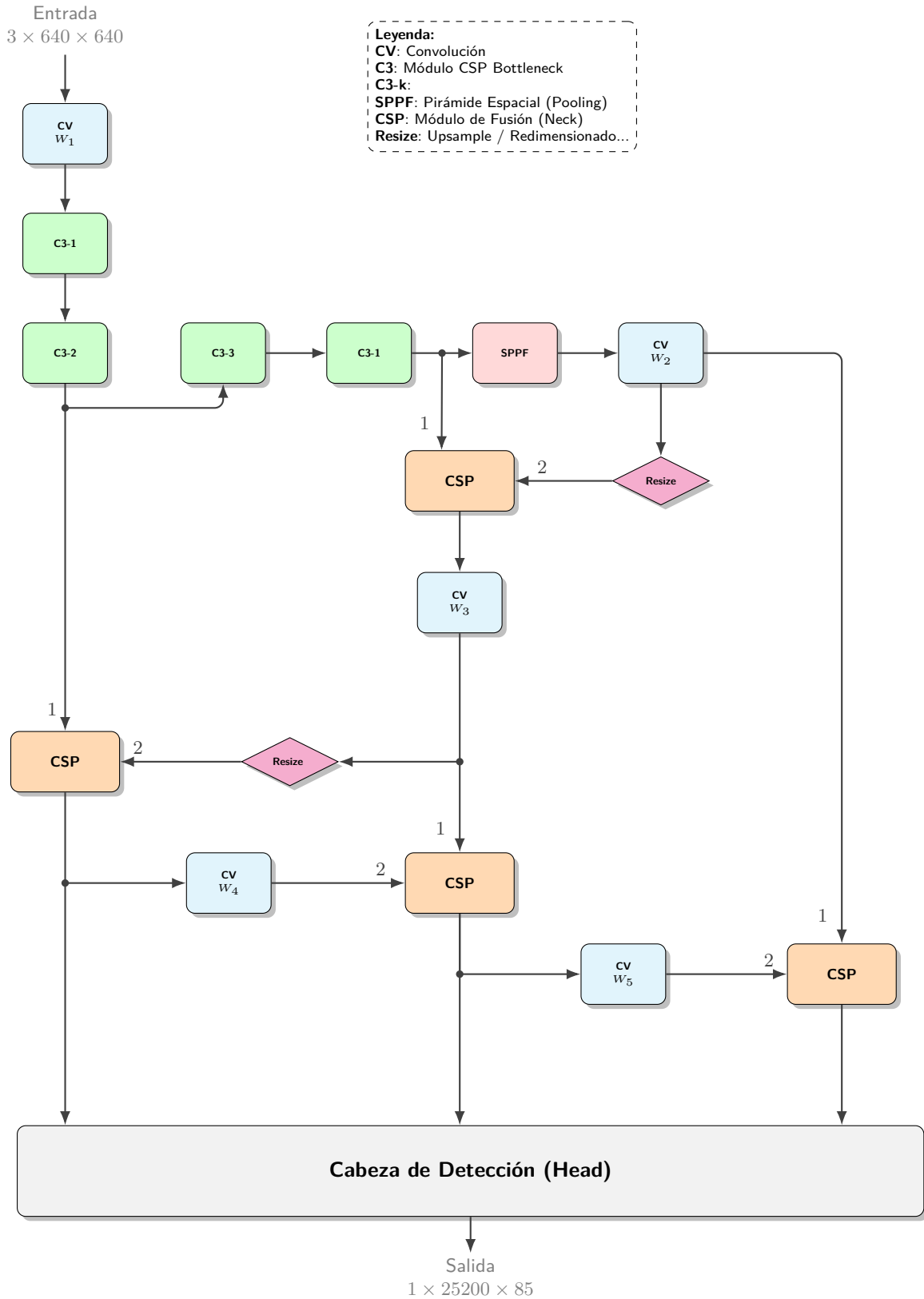


bhuasb uidgashd aysbui dkas, 6dawhjkd yuas



**Figura 9:** Detalle de la Cabeza de Detección (YOLO Head)

La implementación final se obtiene del análisis de la estructura 10



**Figura 10:** Arquitectura Global YOLO: Inicio Vertical y Flujo Descendente

## 2.3. Reglas de entrenamiento

## 3. Desarrollo

## 4. Resultados

## Referencias

- [1] G. Jocher, “YOLOv5 by Ultralytics,” 5 2020.
- [2] V. Dumoulin and F. Visin, “A guide to convolution arithmetic for deep learning,” 2018.
- [3] S. Elfving, E. Uchibe, and K. Doya, “Sigmoid-weighted linear units for neural network function approximation in reinforcement learning,” 2017.
- [4] P. Ramachandran, B. Zoph, and Q. V. Le, “Searching for activation functions,” 2017.
- [5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” 2016.
- [6] J. Redmon and A. Farhadi, “Yolo9000: Better, faster, stronger,” 2016.
- [7] J. Redmon and A. Farhadi, “Yolov3: An incremental improvement,” 2018.
- [8] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “Yolov4: Optimal speed and accuracy of object detection,” 2020.
- [9] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” 2015.
- [10] C.-Y. Wang, H.-Y. M. Liao, I.-H. Yeh, Y.-H. Wu, P.-Y. Chen, and J.-W. Hsieh, “Cspnet: A new backbone that can enhance learning capability of cnn,” 2019.
- [11] K. He, X. Zhang, S. Ren, and J. Sun, *Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition*, p. 346–361. Springer International Publishing, 2014.